PRO2000 - PROGRAMMING

Week 36- Conditionals, Loops

Gebremariam Assres
Gebremariam.assres@kristiania.no

Today's topics

Selection structures

- if
- if-else
- switch

Iteration structures

- for
- while
- do-while

Overview

- The sequence in which instructions are executed is called **program flow**.
- By default, C++ follows a top-to-bottom execution order.
- However, control structures are used to determine the order in which program statements are executed.
- Control structures alter this sequence to support: decision-making, selection, repetition.
- They allow a program to make decisions and perform repeated operations.
- Common C++ control structures include: **if-else, switch, for, while, do-while**

Conditionals

Relational expressions

- Relational expressions are boolean which evaluate to **`true (1)`** or **`false (0)`**
- Examples of expressions, logical operators, precedences:

`Constants:`

`12 > 5` is **`true`**

`7 <= 5` is **`false`**

Variable like if **`x`** is 10, then:

`x == 10` is **`true`**,

`x <= 8` is **`false`**,

`x != 8` is **`true`**, and

`x == 8` is **`false`**

Relational expressions …

- Examples …

```
int x = 12, y = 5, z = -4;
```

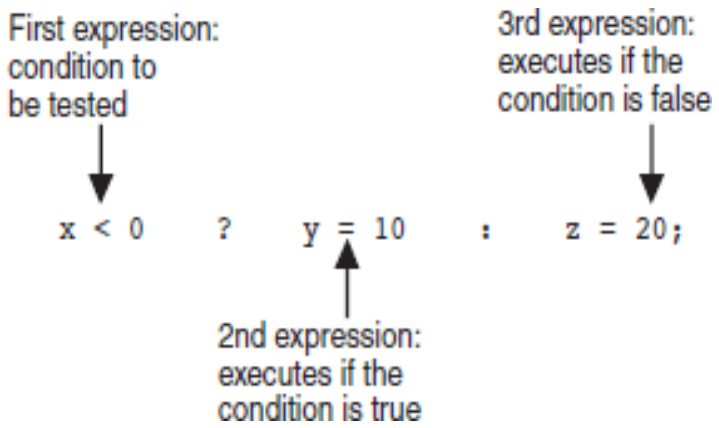
`(x > y) && (y > z)`	`true`
`(x > y) && (z > y)`	`false`
`(x <= z) || (y == z)`	`false`
`(x <= z) || (y != z)`	`true`
`!(x >= z)`	`false`

Relational expressions ...

- Can be assigned to a variable

```
bool result = (x <= y);
```

- Assigns **0** for **false**, **1** for **true**
- **Conditional operator- expr ? expr : expr;**



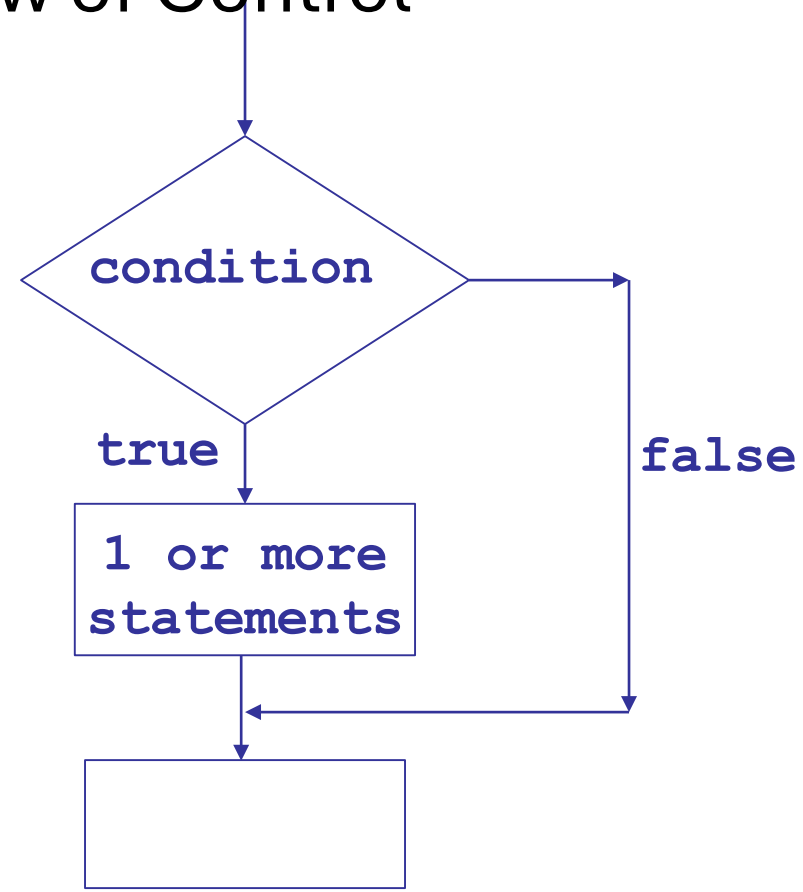
- Do not confuse **=** (assignment) and **==** (equal to)

The **if** Statement

- Supports the use of a decision structure
- Allows statements to be conditionally executed or skipped over
- Models the way we mentally evaluate situations
  - “If it is cold outside,
    - wear a coat and wear a hat.”
- The **if** statement is used to evaluate a condition.
- If the condition is true, one block of code is executed.
- If the condition is false, an alternative block is executed.

The **if** Statement ...

Flow of Control




```
if (condition) {
   // statements if true
}
```

```
if (score >= 60)
     cout << "You passed." << endl;
```

```
if (score >= 90)
{
     grade = 'A';
     cout << "Wonderful job!" << endl;
}
```

The **if** Statement ...

- `if` is a keyword. It must be lowercase
- `(`***`condition`***`)` must be in ( )
- Do not place `;` after `(`***`condition`***`)`
- Don't forget the { } around a multi-statement body

The **if** Statement ...

- Flag
  - A variable that signals a condition
  - Usually implemented as a **`bool` but can also be an `integer`**
  - Meaning:
    - **`true`**: the condition exists
    - **`false`**: the condition does not exist
  - The flag value can be both set and tested with **`if`** statements

```
bool validMonths = true;
…
if (months < 0)
validMonths = false;
…
if (validMonths)
moPayment = total / months;
```

```
int allDone = 0; // set to false …

if (count > MAX_STUDENTS)
   allDone = 1; // set to true …
if (allDone)

   cout << "Task finished";
```

The **if** Statement …

- Nested if statement
  - … an if statement placed inside the body of another if or else block
  - It allows you to test multiple layers of conditions, where the inner condition is evaluated only if the outer condition is true.

```
int age = 20;
bool hasLicense = true;

if (age >= 18) {
        cout << "Age requirement met." << endl;

        if (hasLicense) {
                cout << "You are allowed to drive!" << endl;
        }
}
```

```
float score=95;

if (score < 100)
{
        if (score > 90)
                grade = ‘’A’’
}
cout<<grade<<endl;
```

The **if** Statement ...

- Input validation: inspecting input data to determine if it is acceptable thereby avoiding bad input
- Can perform various tests- range, reasonableness, valid menu choice, zero as a divisor, etc.
  - Used to test if a value is within a range

    ```
    if (grade >= 0 && grade <= 100) cout << "Valid grade";
    ```

  - Can also test if a value lies outside a range

    ```
    if (grade <= 0 || grade >= 100) cout << "Invalid grade";
    ```

- Cannot use mathematical notation

  ```
  if (0 <= grade <= 100) //Doesn't work!
  ```

The **if** Statement ...

- Scope of a variable is the block in which it is defined, from the point of definition to the end of the block
  - Variables defined inside **{ }** have local or block scope
- Variables are usually defined at the beginning of a function
- They may instead be defined close to the place where they are first used
- Can use relational operators with characters and string objects

```
if (menuChoice == 'A')

if (firstName   == "Beth")
```

- Comparing characters is really comparing ASCII values of characters
- Use cctype header to apply functions isalpha, isdigit, islower, isupper, etc.

The **if-else** Statement ...

- **If (condition) is true, statement set 1 is executed and statement set 2 is skipped.**

- **If (condition) is false, statement set 1 is skipped and statement set 2 is executed.**

```
if (condition)
{
      statement set 1;
}
else
{
      statement set 2;
}
```

```
#include <iostream>
using namespace std;
int main() {
    int number = 10;
    if (number > 0) {
        cout << "Positive number" << endl;
     }
     else {
       cout << "Non-positive number" << endl;
     }
     return 0;
}
```

The **if-else** Statement ...

- **`if/else chain`** with Trailing **`else`**

```
if (age   >= 21)
     cout << "Adult";
else if   (age >= 13)
     cout << "Teen";
else if   (age >= 2)
     cout << "Child";
else
     cout <<  "Baby";
```

The **switch** Statement

- The **switch** statement enables multi-way selection to compare an expression against a set of constant case values.
- It is often preferred over multiple **if-else if** statements when testing one variable.
- IntExpression must be a char or an integer variable or an expression that evaluates to an integer value
- exp1 through expn must be constant integer type expressions and must be unique
- default is optional but recommended
- Use break; statement to stop execution in the current block or to exit a switch statement

```
switch (IntExpression)
{
   case exp1: statement set 1;
              break;
   case exp2: statement set 2;
              break;
               ...
   case expn: statement set n;
              break;
   default: statement set n+1;
}
```

The **switch** Statement ...

Example of **switch**

```
#include <iostream>
using namespace std;
int main() {
    int choice = 2;
    switch (choice) {
        case 1:
            cout << "Choice is 1" << endl;
            break;
        case 2:
            cout << "Choice is 2" << endl;
            break;
        default:
            cout << "Invalid choice" << endl;
            break;
    }
    return 0;
}
```

The **switch** Statement …

- **`switch`** is a natural choice for menu-driven program
  - display menu
  - get user input
  - use user input as **`IntExpression`** in **`switch`** statement
  - use menu choices as **`exp`** to test against in the **`case`** statements
- Use enumerated types in switch statements
  - Format: enum class name {val1, val2, … valn};
  - Examples:
    - enum class Fruit {apple, grape, orange};
    - enum class Days {Mon=1, Tue, Wed, Thur, Fri}

Loops

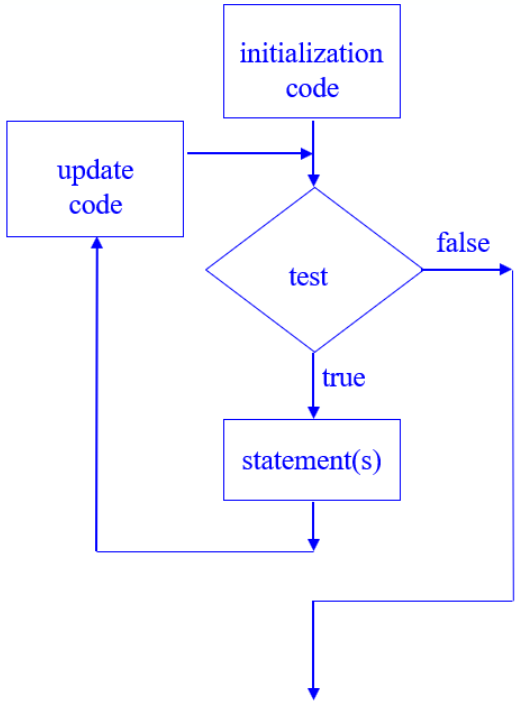
The **for** Loop

- The **for** loop is used when the number of iterations is known in advance.
- It combines initialization, condition checking, and update in a single statement.
- It is widely used for counting operations.
- Use counter variables, sentinels, nested loops
- Initialization and update code can contain more than one statement separated by commas

```
for (int sum=0, num=1; num<=10; num++) sum+=num;
```

The **for** Loop …

```
for (initialization; condition; update)
{
   // repeated statements
}
```



The **for** Loop …

Exampl 1

```
#include <iostream>

using namespace std;

int main() {

   cout << "For Loop Output: ";

   for (int i = 0; i < 5; i++) {

      cout << i << " ";

   }

   cout << endl;

   return 0;

}
```

Exampl 2

```
int sum = 0, num;

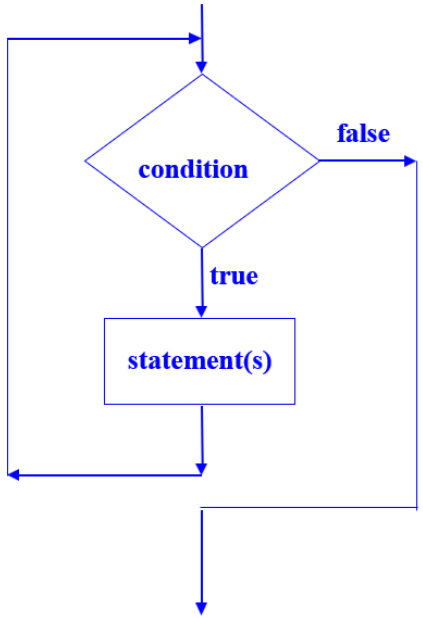
for (num = 1; num <= 10; num++)
        sum += num;

cout << "Sum of numbers 1 – 10 is "
        <<sum<<endl;
```

The **while** Loop

- The **while** loop repeats a block of code as long as a condition remains true.
- It is suitable when the number of iterations is not known beforehand.
- The condition is tested before the loop body is executed.

```
while (condition) {
   // repeated statements
}
```



The **while** Loop …

Example of **while** Loop

```
#include <iostream>
using namespace std;
int main() {
    cout << "While Loop Output: ";
    int count = 0;
    while (count < 5) {
        cout << count << " ";
        count++;
    }
    cout << endl;
    return 0;
}
```

```
int val = 5;
while (val >= 0)
{    cout << val << "   ";
     val = val - 1;
}
```

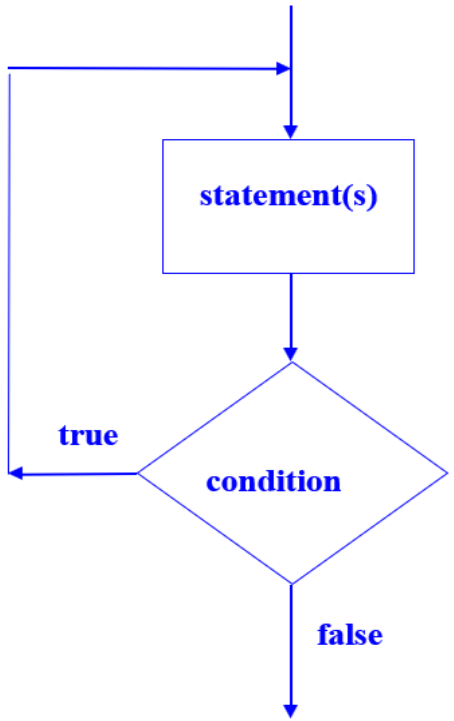
produces output:

```
  5   4   3   2   1   0
```

The **do-while** Loop

- The **do-while** loop is similar to the **while** loop.
- However, the condition is evaluated after the loop body executes.
- Therefore, the loop body always executes at least once.

```
do {
   // repeated statements
} while (condition);
```



The **do-while** Loop …

Example of **do-while** Loop

```
#include <iostream>
using namespace std;
int main() {
    cout << "Do-While Loop Output: ";
    int num = 0;
    do {
        cout << num << " ";
        num++;
    } while (num < 5);
    cout << endl;
    return 0;
}
```

Menu-Driven Program Example

```
do {
    • // code to display menu
    • // and perform actions
    cout << "Another choice? (Y/N) ";
} while (choice =='Y'||choice =='y');
```

- The condition could be written as
  - `(toupper(choice) == 'Y');`
- or as
  - `(tolower(choice) == 'y');`

Summary

- Elements of Conditional Structures
  - **Condition-** A logical expression that evaluates to true or false
  - **Execution block-** The statements performed when a condition is satisfied
  - **Alternative block-** The statements performed when a condition is not satisfied
  - **Case labels-** Specific constant values used in a **switch**
  - **Default case-** Executed when no case label matches

Summary …

- Elements of Iteration Structures
  - **Initialization-** Establishes the starting state of the loop
  - **Condition-** Determines whether another iteration should occur
  - **Loop body-** Contains the statements to be repeated
  - **Update-** Modifies the loop variable or state
  - **Termination-** Occurs when the condition evaluates to false

Up next: Exercises

- Now: 2 hours (12:15 to 14:00) of exercises (found on Canvas).

- Short exercise summary:
  - *Write, compile, and debug a C++ programs with emphasis on statements formed by if, if-else, switch, for, while, and do-while*
- Remember:
  - You should do approximately 4 more hours of the course (in a study group or alone) *before* our next session together. :-)